Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 08 - Part 2

Rule of 3, Pointers to Objects, Inheritance

THIS WEEK'S AGENDA

01. Rule of 3
02. Pointers to Class Objects
03. Inheritance
04. Derived Classes
05. Access Specifiers

Rule Of 3

THE RULE OF 3

- The rule of 3 is another design principle that states that if a class requires a user-defined destructor, a user-defined copy constructor, or a user-defined copy-assignment operator then it almost certainly requires all three.
- Recall that the default copy constructors and assignment operators create a shallow copy.
- Therefore, if a class has a user-defined copy constructor and assignment operator then that means we want to create a deep copy of our objects.

THE RULE OF 3

- Suppose that our class does not define a copy constructor. This means that copying an object will copy all of its data members to the target object.
- When the time comes to destroy the original object, the destructor will actually be invoked twice, and will have the same information for each object (original and copied).
- Therefore, in the absence of an appropriately defined copy constructor, the destructor is executed twice when it should actually only execute once.
- This duplicate execution can become a problem and lead to memory leaks or unwanted object deletions.

THE RULE OF 3 - DESTRUCTORS

- Recall that a **destructor** is a function that is
  - called when an object goes out of scope or,
  - is explicitly deleted.
- It is responsible for cleaning up resources that the object may have acquired during its lifetime, such as dynamically allocated memory.

THE RULE OF 3 - COPY CONSTRUCTORS

- A constructor that creates a new object as a copy of an existing object.
- It is called when an object is passed by value, returned from a function, or explicitly copied.
- If the class manages resources (like dynamic memory), a **default copy constructor**, will lead to **shallow copies**, causing resource management issues.
- What is a shallow copy? It is creation of an alias of a dynamically allocated object.

THE RULE OF 3 - COPY ASSIGNMENT OPERATOR

- This is an overloaded = operator.
- It is used to copy the contents of one existing object to another existing object.
- Like the copy constructor, if the class manages resources, the **default assignment operator**, will lead to shallow copies.

COPY CONSTRUCTOR VS COPY ASSIGNMENT OPERATOR

- **Copy Constructor:** A special constructor that initializes a new object as a copy of an existing object.
  - `ClassName(const ClassName &other);`
  - Called when a **new object** is created from an existing object.
- **Copy Assignment Operator:** A special assignment operator that assigns the contents of one existing object to another existing object (both objects are already created).
  - `ClassName& operator =(const ClassName& other);`
  - Called when an **existing object** is assigned to another existing object.

RULE OF 3 - EXAMPLE

```cpp
#include <iostream>
#include <cstring>

class MyString {
    private:
        char* data;
    public:
        // Constructor
        MyString(const char* str = "") {
            data = new char[strlen(str) + 1];
            strcpy(data, str);
        }

        // Destructor
        ~MyString() {
            delete[] data;
        }

        // Copy Constructor
        MyString(const MyString &other) {
            data = new char[strlen(other.data) + 1];
            strcpy(data, other.data);
        }

        // Copy Assignment Operator
        MyString& operator =(const MyString &other) {
            if (this == &other) {
                return *this; // handle self-assignment
            }
            delete[] data; // free existing resource

            data = new char[strlen(other.data) + 1];
            strcpy(data, other.data);
            return *this;
        }

        void print() const {
            std::cout << data << std::endl;
        }
};
```

RULE OF 3 - EXAMPLE

```
42  int main() {
43      MyString str1("Hello, World!");
44      MyString str2 = str1; // Calls copy constructor
45      MyString str3;
46      str3 = str1; // Calls copy assignment operator
47
48      str1.print();
49      str2.print();
50      str3.print();
51
52      return 0;
53  }
```

RULE OF 3 - EXAMPLE

- Some of the problems that can occur when the rule of 3 is not followed are:
  - **Double Deletion:** deleting an object that has already been deleted due to the default copy constructor/copy assignment operator behavior.
  - **Shallow Copy:** creating shallow copies of objects, leading to a modification in all of them when only one of them is modified.
  - **Memory Leaks:** improper deletion and release of resources may cause memory leaks.

Pointers To Class Objects

THE → NOTATION

- In C++, the → symbol is called the arrow operator and is primarily used to access members (attributes/data or methods) of an object or a struct through a pointer.
- When you have a pointer to an object or a struct, instead of dereferencing the pointer and then accessing the member variable or function using the dot (.) operator, you can instead use the → operator.
- Lets say we have the Node class with the member variable called value, as shown in the previous slide.

THE → NOTATION

- Then, if you have a pointer to a Node object, you can access its members as follows:
  - `Node *node1 = new Node;`
  - `node1->value = 15;`
  - `std::cout << “Node1 value: “ << node1->value << std::endl;`
- Without using the arrow operator, this would be:
  - `(*node1).value = 15;`

Inheritance

INTRODUCTION TO INHERITANCE

- In object-oriented design, **inheritance** is a relationship between a more general class (called the **base class**) and a more specialized class (called the **derived class**).
- The **base class** is sometimes also called the **superclass** or **parent class**, while the **derived class** is also called the **subclass** or **child class**.
- The derived class inherits data and behavior from the base class.
- Inheritance can be looked at as an “is-a” relationship between a general base class and a specialized derived class.

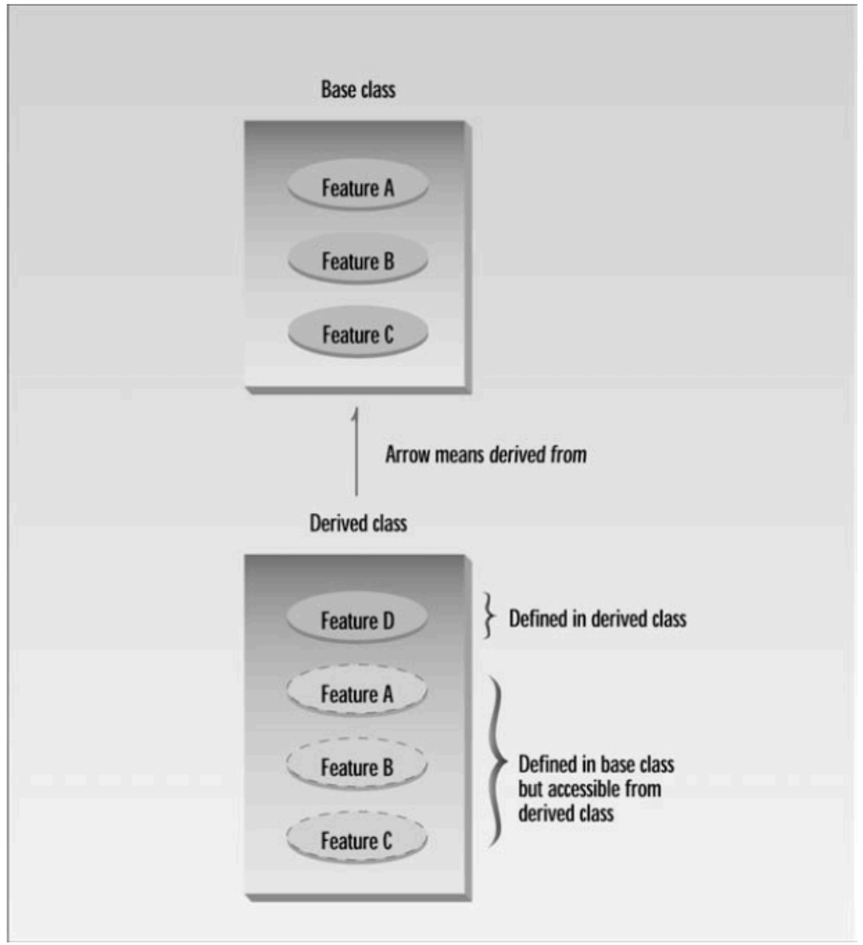
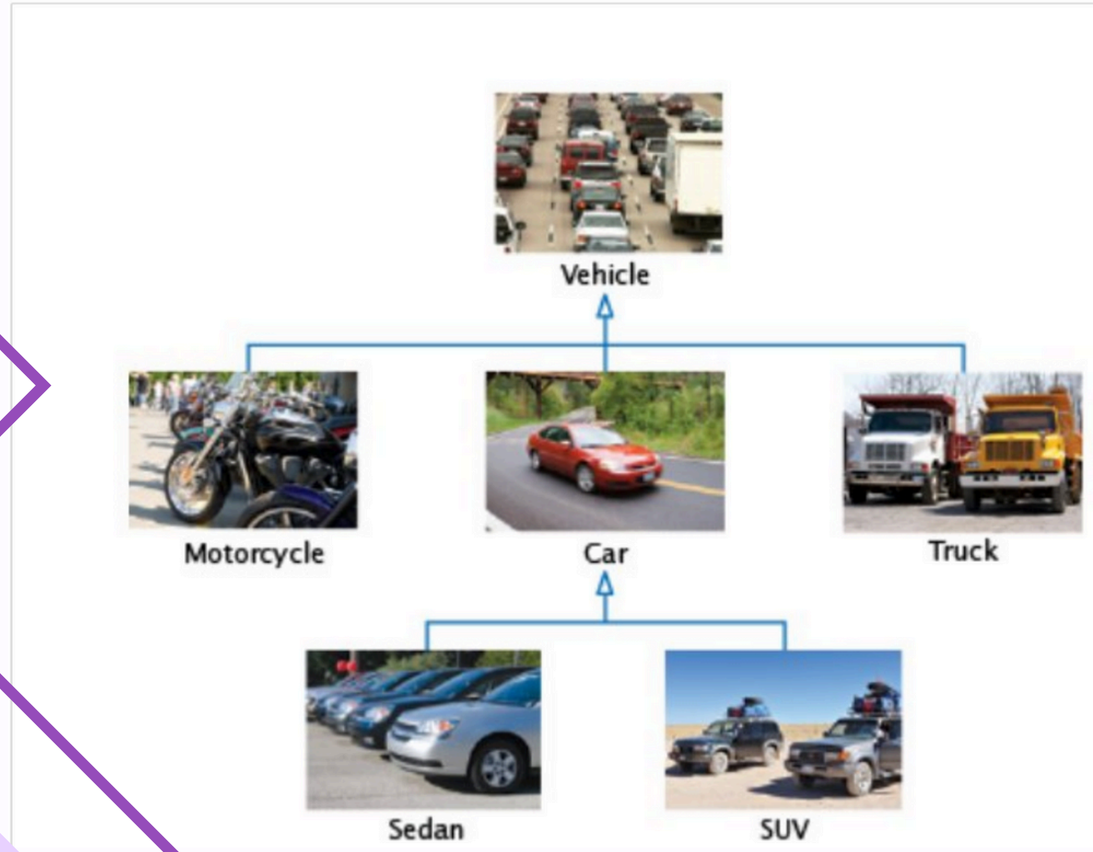


FIGURE 9.1
Inheritance.

THE INHERITANCE HIERARCHY

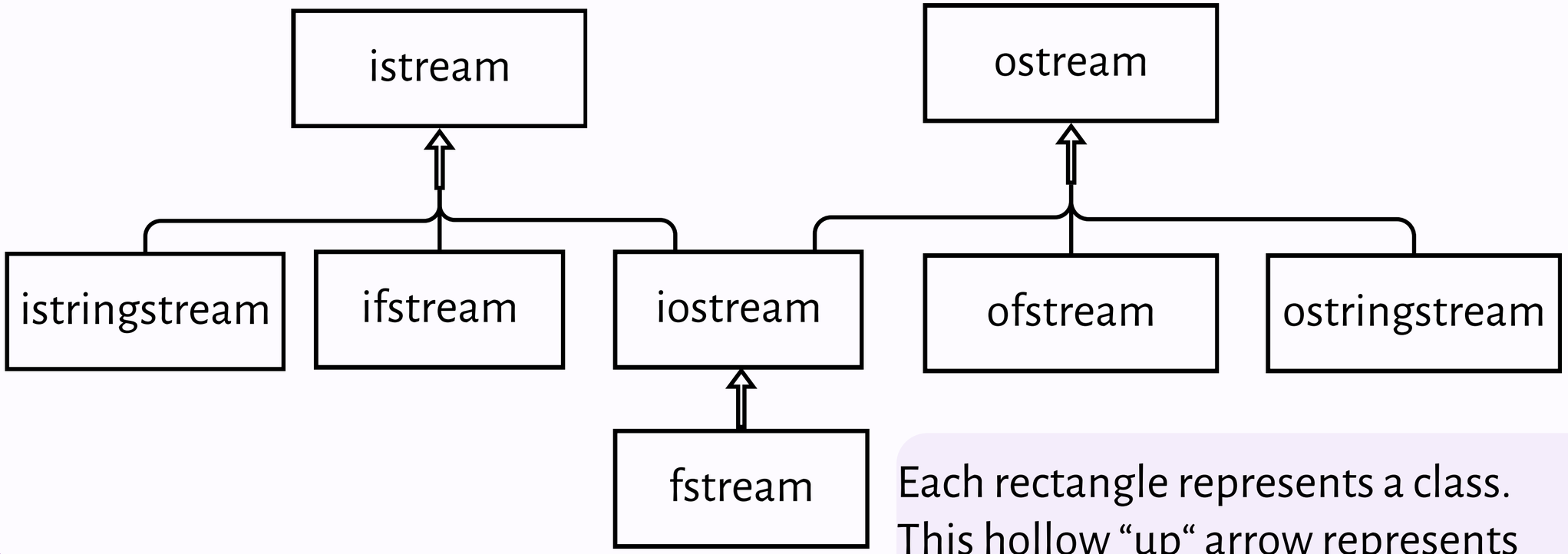


- Cars share common traits of vehicles, such as the ability to transport.
- The class Car inherits from the class Vehicle.
- The Vehicle class is the base class in this case and the Car class is the derived class.

INTRODUCTION TO INHERITANCE

- The inheritance relationship is very powerful because it allows us to reuse algorithms with objects of different classes.
- Suppose we have an algorithm that manipulates a Vehicle object.
- Because a *car* is a special *vehicle*, we can supply a Car object to such an algorithm, and it will work correctly.
- This is an example of the **substitution principle** that states that you can always use a derived-class object when a base-class object is expected.

INHERITANCE HIERARCHY OF C++ IO CLASSES

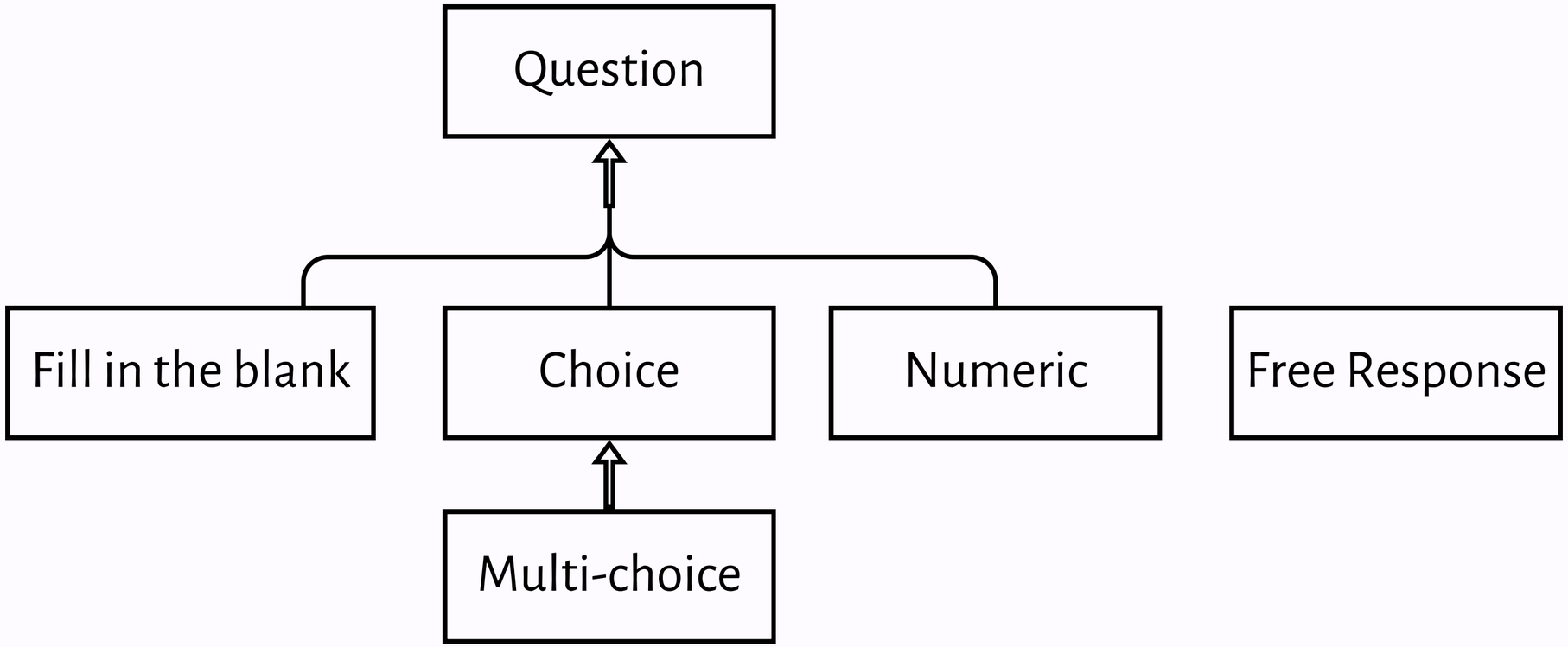


Each rectangle represents a class.
This hollow “up“ arrow represents the inheritance relationship.

QUESTION HIERARCHY - EXAMPLE

- Consider a simple hierarchy of questions in a quiz. A quiz consists of different kinds of questions such as:
  - Fill in the blanks
  - Choice (single or multiple)
  - Numeric
  - Free responses
- An inheritance hierarchy for these question types can be constructed.

QUESTION HIERARCHY - EXAMPLE



Derived Classes

DERIVED CLASSES

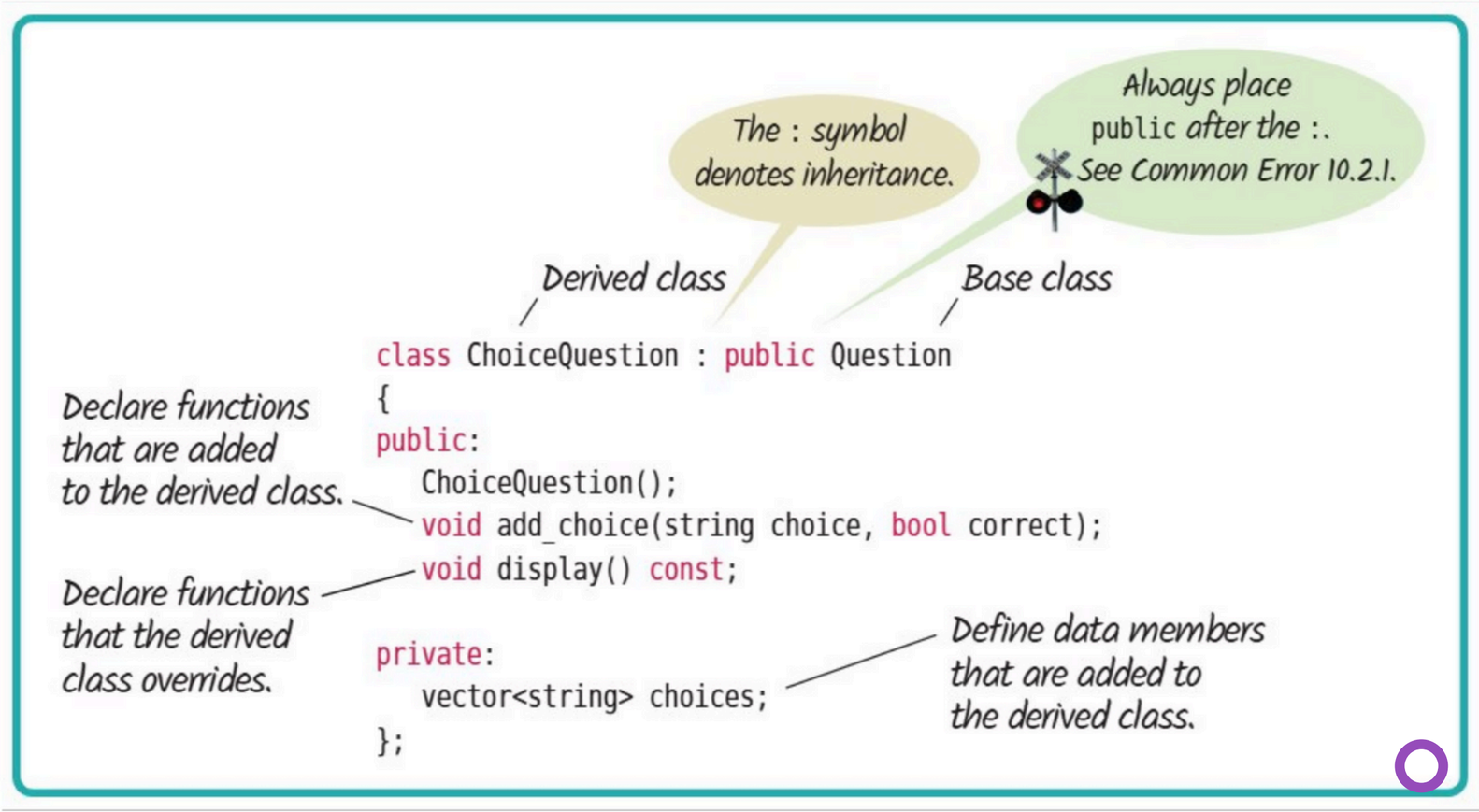
- In C++, we form the derived class from the base class by specifying what makes the derived class different.
- We also define the member functions that are new to the derived class.
- The derived class inherits all member functions from the base class.
- However, we can change the implementation of the inherited member functions as per our need.
- The derived class automatically inherits all data members from the base class.
- We only have to define the additional data members.

DERIVED CLASSES – EXAMPLE

- The syntax of a derived class is as follows:

```
class Child : public Parent {
   public:
      New and changed member functions
   private:
      Additional data members
};
```

DERIVED CLASSES – FORMAT



Access Specifiers

ACCESS SPECIFIERS

- So far, we have been looking at two access specifiers: **public** and **private**.
- However, there is a third access specifier as well - **protected**.
- Let's look at each in detail:
  - **public –** class members declared as public can be accessed anybody.
  - **private –** class members declared as private can only be accessed by member functions of the same class or friends. ***This means that derived classes can not access private members of the base class directly.***
  - **protected –** class members declared as protected can be accessed by the class itself, friends, and also derived classes. They are not accessible from outside the class.

Summary

SUMMARY

- We discussed two design principles: RAII and Rule of 3
- The Rule of 3 states that: if a class needs to define one of the following:
  - Destructor, Copy Constructor, Copy Assignment Operator, then
- it needs to explicitly define all 3 in order to have exception-safe coding.
- Without Rule of 3, problems such as:
  - Double Deletion, Shallow Copies, and Memory Leaks
- may happen.
- Introduction to inheritance included the definitions of a base class and derived class, and the syntax of a derived class in C++.

READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 08 and

ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science